The pair HMM

What it computes, how it differs from the HMMs you have probably met before, and how the current code is built.

This is written to be read from the start with no prior context. Section 1 treats the model as a black box and says only what goes in and what comes out. Section 2 explains why this kind of HMM and not the familiar profile HMM. Section 3 opens the box. Sections 4 onward are the definitions and the code. Every number quoted was measured on the code as it stands.

1. The black box

Before any detail, here is the whole thing from outside. It is one function that answers one question:

Given that this read came from this exact place in this assembly, how probable was it to look the way it does?

1.1 What goes in

```
input            type                  where it comes from
--------------   -------------------   -------------------------------------
read             string over ACGT      the sequenced read, as sequenced
qualities        one Phred int per     the read's quality string, same order
                 read base
template         string over ACGT      the assembly sequence starting at the
                                       candidate position, on the relevant
                                       strand
parameters       5 numbers             gap-open and gap-extend probabilities;
                                       fixed, shared by every read and both
                                       assemblies
```

That is all. There is no database, no alignment record, no CIGAR, no BAM. Four arguments, three of which are just sequence and numbers.

Two things about the inputs are worth stating now because they are easy to get wrong later.

The **read is never reverse-complemented**. If the read came off the reverse strand, it is the *template* that is reverse-complemented, because the strand is a property of the place the read came from, not of the read. Section 4.3 shows this is not merely a convention.

The **template starts at the candidate position**. It is not "the contig" and not "the region the aligner reported". It is the assembly sequence read forward from one specific base, continuing until the recruited segment ends.

1.2 What comes out

One number:

```
logp    log P(read | this position, this assembly)
```

plus two diagnostics that say whether to trust it:

```
terminal    where in the template the read's alignment ended, as a
            distribution rather than a single answer
edge_mass   the fraction of that distribution sitting at the template's
            right edge -- i.e. how much the read wanted to run off the end
```

1.3 A worked example

A 10-base read against a 10-base template, quality 40 throughout:

```
read      ACGTACGTAC
template  ACGTACGTAC        ->  logp = -0.002873

read      ACGTTCGTAC        (one base changed)
template  ACGTACGTAC        ->  logp = -10.310800
```

The difference is 10.308 nats. That is exactly `s`, the cost of one substitution at Q40 (section 3.3). Nothing else moved, so nothing else was charged.

1.4 What the number means, and what it does not

It is a **log probability of the read**, conditional on the position. Probabilities of a specific 15,000-base string are astronomically small, so the number is large and negative for any real read; the perfect-match example above is near zero only because the read is ten bases long.

Absolute values carry no information. Only differences do — between two positions, or between two assemblies. Every comparison the method makes is a difference, so the common scale cancels.

It is **not an alignment score**. That distinction matters for two concrete reasons:

- Scores from different positions cannot be added. Probabilities can, and we need to add them (section 1.5).
- Scores from different assemblies cannot be divided. Probabilities can, and the whole method is one such ratio.

An aligner's bit score or edit distance fails both tests. That is the real reason this exists.

1.5 How it is used: call it once per position, then add

The black box answers for **one** position. The quantity the assembly comparison actually needs is the sum over every position in the locus:

```
per_position = [logp(read, qual, template_at(p)) for p in positions]
total        = logsumexp(per_position)
```

Here is that sum on a real run. A 30 bp unit, and a template holding two exact copies of it, so the read genuinely fits in two places:

```
position  0   ->  -0.008873      a real copy
position 30   ->  -0.008873      the other real copy
position  7   -> -26.501         nonsense, contributes nothing
...
sum over all positions  =  +0.684304
same read, one copy only=  -0.008873
difference              =  +0.693177        log 2 = 0.693147
```

Two copies score `log 2` above one copy. That is not a coincidence or an approximation — it falls out of adding two nearly equal terms instead of one.

This is the entire copy-number signal of the method. An assembly that collapsed a 3-copy array to 1 copy loses `log 3` per read that sits in the array. Nothing more sophisticated is happening: the signal is arithmetic on a sum.

(The tiny excess over exactly `log 2` is real and explained in section 6.1.)

2. Why a pair HMM and not a profile HMM

If you have used HMMs in bioinformatics before, it was almost certainly a **profile HMM** — HMMER, Pfam, hmmsearch. This is a different animal, and the difference is worth being precise about, because the intuitions do not transfer.

2.1 A profile HMM in one paragraph

A profile HMM represents a *family* of sequences. You build it from a multiple alignment of many known members. For each consensus column of that alignment it creates a match state with **its own emission distribution** — column 12 might strongly prefer cysteine, column 13 might accept anything — plus per-column insert and delete states. A 300-column protein family gives roughly 900 states and several thousand fitted emission probabilities. To use it, you run a query sequence against the model and ask how well it scores.

The pattern lives inside the model. The model *is* the family. That is why building one requires training data, and why the model is large.

2.2 A pair HMM in one paragraph

A pair HMM represents a *relationship between two sequences*: the process by which one could be a corrupted copy of the other. It has **three states, always**, no matter how long the sequences are. The match state does not have a preferred letter; it emits a *pair*, and all it cares about is whether the two letters agree. Fix one sequence as given, and the model becomes a probability distribution over the other.

The pattern lives in the input. The template is an argument, not part of the model.

2.3 Side by side

```
                     profile HMM                pair HMM (this one)
-------------------  -------------------------  -------------------------
what it models       a family of sequences      a copying process
number of states     ~3 per consensus column    3, always
emission depends on  which column you are in    whether the two bases agree
                                                and the read's quality
parameters           thousands, fitted from     5 gap probabilities, plus
                     a multiple alignment       Phred, which is not fitted
needs training data  yes, an MSA of the family  no, only an error model
the other sequence   baked into the states      passed in as an argument
typical question     does this belong to the    how likely is this read to
                     family?                    be a copy of this template?
```

2.4 Why the profile architecture would be hopeless here

Not merely inconvenient — unusable, for a structural reason.

The template changes at every position we score. Each candidate origin gives a different template. A profile HMM would have to be rebuilt for each one, and each rebuild would need training data that does not exist. The pair HMM takes the template as an argument, so

scoring a new position is a new function call with the same model.

There is also nothing to train on. A profile HMM learns which columns are conserved from many aligned family members. Here there is exactly one template and one read, and the only thing we know about their relationship is the sequencer's error behaviour — which is five numbers, shared genome-wide.

2.5 The one profile-HMM feature we deliberately do not have

HMMER's architecture includes flanking states (`N` and `C` in Plan7) whose job is to absorb query residues *outside* the matched region, at low cost. That is what makes local alignment work: a query can have arbitrary junk on either side of the hit and pay almost nothing for it.

We have no such state. Every read base must be emitted by `M` or `I`. There is no way to say "these 500 bases are outside the region of interest, ignore them".

The consequence is concrete and measurable. If a read extends past the end of its template, the overhanging bases have to be explained as insertions:

```
bases hanging past the template end     logp
-----------------------------------     --------
   0                                       -0.018
   1                                     -10.536
   5                                     -20.895
  20                                     -58.913
```

About 2.6 nats per overhanging base after the first. A read hanging 500 bases off the end would score around -1300 nats — not impossible, but so improbable that it would dominate any comparison it took part in, for a reason that has nothing to do with which assembly is right.

This is why the containment rule exists. Reads are only scored if they fall entirely inside a recruited segment. That rule is often described as a nuisance that throws away a third of the data at a 45 kb locus; it is better understood as the thing that stops boundary reads from being scored by a model that cannot represent them. It is also why the exposure term uses an effective length `L_eff` rather than the raw segment length: if you discard reads near segment ends, you must also discard the positions they would have come from.

The alternative — adding flanking states — is a real design option and not obviously wrong, but it would change what the score means and is not something to do casually.

2.6 What we do borrow

The **free start** and **free end** of section 5 are the same idea as local alignment, applied on the template side instead of the query side. A profile HMM lets the query hang off both ends of the model; we let the template hang off both ends of the read. Trailing template costs nothing, and leading template is handled by summing over start positions.

And there is one place the two architectures may yet converge. A profile HMM's strength is *position-specific* parameters. The natural next step for this model (section 7.2) is to make the gap probabilities depend on the local template context — high in homopolymer runs, low elsewhere. That is position-specificity too, but derived from the template sequence by a rule rather than fitted per column, so it stays five-ish parameters instead of thousands.

3. The model as built

3.1 The idea in one paragraph

A read is a corrupted copy of some stretch of the assembly. Imagine the sequencer walking along the template and the read together, one step at a time, and at each step doing one of three things: copying a base (possibly wrongly), inserting a base that is not in the template, or skipping a template base. Every complete walk is one explanation of how this read could have arisen. There are many such walks, we do not know which one happened, so we add up the probabilities of all of them. That total is the probability of the read.

3.2 Three states

States are distinguished by **what they consume**:

```
state   consumes read?   consumes template?   meaning
-----   --------------   ------------------   ------------------------------
  M          yes                yes           aligned column (match/mismatch)
  I          yes                no            insertion: base in read only
  D          no                 yes           deletion: base in template only
```

There is no separate mismatch state. A match and a mismatch consume exactly the same thing and differ only in how likely the emitted letter is, so folding them together is exact.

3.3 Transitions and emissions

```
from -> to    probability              meaning
----------    -----------------------  ---------------------
B -> M        1 - dI                   begin aligned
B -> I        dI                       begin with an insertion
M -> M        1 - dI - dD - tau        keep matching
M -> I        dI                       open an insertion
M -> D        dD                       open a deletion
M -> E        tau                      terminate
I -> I        gI                       extend insertion
I -> M        1 - gI - tau             close it
I -> E        tau                      terminate
D -> D        gD                       extend deletion
D -> M        1 - gD                   close it
```

There is no edge between `I` and `D` in either direction, so a deletion can never directly follow an insertion. Two further edges are absent on purpose: section 3.6.

For emissions, with `eps = 10^(-Q/10)` the error probability of the read base, state `M` emits the observed base with probability `1 - eps` if it matches the template and `eps/3` if it does not. The cost of one substitution is the log-ratio:

s = log(3(1-eps) / eps)

which is 5.7 nats at Q20, 8.0 at Q30 and 10.3 at Q40. The reads in this project average Q39, giving **s = 10.08**.

State `I` emits a base with probability 1/4. State `D` emits nothing.

Using the read's **own** per-base qualities rather than a flat rate is not a refinement. In a repeat array where a handful of variant columns separate the two candidate assemblies, it is the difference between a read that decides the question and a read that does not.

3.4 What the gap parameters do and do not model

This deserves its own section because it is the easiest thing to misread.

`dI, dD` open a gap; `gI, gD` extend one. Separating them makes gap lengths geometric with mean `1/(1-g)`. At the current placeholder `g = 0.3` that is 1.43 bases — appropriate for HiFi, where indel errors are overwhelmingly one-base homopolymer slips.

They model **sequencing error, not real sequence difference.** That distinction drives the numbers:

```
deletion of k bases     pair HMM      exp(-NM)
-------------------     ---------     ---------
     1                    -8.47         -1.00
     2                   -10.77         -2.00
     5                   -13.28         -5.00
    20                   -32.43        -20.00
    50                   -67.85        -50.00
```

Note that the pair HMM charges **more** than `exp(-NM)`, not less, for every length. An earlier version of this document claimed the opposite — that affine gaps make long indels cheap. That is only true when `g` is near 1. At a `g` calibrated to HiFi error, a 50 bp deletion is not a plausible sequencing error at all, and the model says so loudly.

That loudness is the signal. A 50 bp deletion between read and template is a real difference. The model reporting -67.85 nats means "this read did not come from this template", which is exactly the verdict we want it to reach. What `exp(-NM)` gets wrong is not the sign but the scale: it under-charges everything, substitutions by a factor of ten, so it discriminates far less sharply than it should.

3.5 Why we add up all the walks

This is the single most important modelling choice, and a concrete case makes it obvious.

Take the template `AAAAAAA` (seven A's) and the read `AAAAAA` (six). One base was deleted. Where? Nothing in the data says. Under the transition graph the deletion can sit at template positions 2 through 6 — five places, all with identical probability, because each such walk uses exactly four `M -> M` transitions regardless of where the deletion falls.

The best single walk (Viterbi) reports the probability `q` of one of them. The sum over walks (forward) reports `5q`. The forward answer is correct: the read really is five times more likely than any single explanation suggests.

This matters because the discrepancy is **different between the two candidate assemblies** — they have different homopolymer run lengths, so different multiplicities. Using Viterbi would inject a bias directly into the quantity being measured.

It is also why the aligner's CIGAR cannot be reused: a CIGAR is one path. **Alignment locates candidate positions; it must never score them.**

A correction to `model.tex`: Remark 6 gives this example and says seven placements, hence log 7. Under the model's own forbidden transitions it is five, hence log 5 = 1.61. The argument is unaffected — Viterbi still understates — but the number should change. The code asserts 5 exactly.

3.6 The two forbidden transitions

Both exclusions remove redundancy, not real possibilities.

No `D -> E`. A walk ending with a deletion consumes template after the last emitted read

base. No observation can distinguish that from having stopped earlier, so allowing it would count one outcome twice.

No `B -> D`. A walk that begins by deleting `t1` and then matching is the same alignment as one that begins a position later. A classical pair HMM aligning two given sequences from a fixed start has no such redundancy and does allow `B -> D`. Here the redundancy exists because we sum over starting positions.

The intuition runs the wrong way here, so it is worth being blunt. When you give the model extra template on the left and right — as we do, to allow for uncertainty about where the read starts — it is tempting to add `B -> D` so the walk can cross the padding. It must not. The padding is already handled by summing over start positions, and adding `B -> D` on top would count "begin at v, delete t_v" and "begin at v+1" as two explanations of one alignment. **Padding makes both exclusions more necessary, not less.**

3.7 Read length and the termination probability

`tau` ends the walk. Formally it is what makes the model a distribution over reads of *variable* length; without it the forward sum is an alignment score and nothing more.

But `tau` implies geometrically distributed read lengths with mean `1/tau`, and that is false for HiFi. A HiFi read stops because the size-selected SMRTbell insert ran out, not because a per-base stopping process fired. The model document concedes this and then discards the implied distribution anyway, by conditioning on the observed length.

The code therefore defaults to **`tau = 0`**: length is conditioned on, not generated. The forward pass returns

log sum_m [f_M(L, m) + f_I(L, m)]

with the termination factor left off. That is legitimate because the factor is *uniform*: every walk emitting L bases terminates exactly once, from M or from I, at probability `tau` either way, so it factors out front.

What does not factor out is the `tau` inside `M -> M` and `I -> M`, since walks with different indel counts carry different powers of those. Measured on a 15 kb read that is a -1.5 nat shift, but only -0.01 nats of *difference* between two candidates disagreeing by 100 indels. Common mode, cancels.

`tau` is a normalisation device, not a parameter. It must never be fitted to observed read lengths.

4. Origins, templates, and the sum

Pedantic on purpose. Most ways to get this wrong are ways of blurring two things that look similar.

4.1 The definitions

Unit `U_A` — a finite set of disjoint intervals of assembly A (the recruited segments), possibly on different contigs. Total length `L_A`.

Origin `p = (c, s, o)` — a contig, a 0-based position, and a strand `o` in `{+, -}`, with `s` inside a segment of `U_A`. The set of origins has exactly `2 * L_A` members — the same `2L_A` that appears in the uniform prior `1/(2L_A)` and in the exposure term `-2 * lambda * L_A`.

Template at `p` — a function of `p` alone. With `[b, e)` the segment containing `s`:

o = + t = A[c][s : e] o = - t = revcomp(A[c][b : s+1])

Truncated at the segment boundary, so an origin near an edge gets a short template. That truncation *is* the containment problem of section 2.5; it is not a separate issue.

Mark `Y = (r, Q)` — the read as sequenced, qualities in sequencing order, never reverse-complemented.

`e(Y | p, A)` — one forward pass of the read against that origin's template. One number.

4.2 What an alignment is, and is not

An alignment is **not** part of any definition above. Alignments are a device for finding origins that carry non-negligible mass, so we need not evaluate all `2 * L_A` of them.

Two consequences, both with teeth:

An origin the aligner never reported is **still a term in the sum**. If `minimap2 -N 20` dropped the 21st copy of a repeat, the model still counts it. Since the copy-number signal *is* the count of origins, truncating the sum at `-N` attacks precisely the quantity being measured.

Having an alignment confers **nothing**. The origin is evaluated like any other. In a repeat most reported placements carry nearly equal mass, and that near-equality is the signal.

4.3 Strand belongs to the template

Because the origin carries the strand, it is the template that gets reverse-complemented; the read stays as sequenced.

Reverse-complementing the read instead is not equivalent, and this was measured rather than assumed: it agrees to 1e-6 nats on an ordinary read, but differs by **0.25 nats** when the read begins with an insertion. The reason is that `B -> I` costs `dI` while `B -> M` costs `1 - dI`, and reversing a walk swaps which end pays which.

5. The code

5.1 Layout

```
src/qamodel/
  __init__.py
  phmm/
    __init__.py
    params.py       transition and emission parameters   [done]
    dp.py    forward and Viterbi                  [done]
```

`qamodel` imports nothing from `qatool`. The dependency runs one way only, `qatool.plotters.likelihood -> qamodel`, which is what keeps the model core usable on simulated sequence with no database, recruiter or BAM involved. One grep checks it.

5.2 params.py

Two kinds of parameter, because they behave differently. Gap and termination behaviour is a property of the chemistry, shared by every read: five numbers. Substitution behaviour is per base, read off the read's own quality string.

`PhmmParams` holds `dI, dD, gI, gD, tau`. It is frozen, because the model requires parameters held fixed across both candidates — fitting them per candidate would let a worse assembly buy itself looser error rates. Immutability means doing that requires visibly constructing a second object.

Its validation exists for one specific failure mode. `M -> M` is `1 - dI - dD - tau` and `I -> M` is `1 - gI - tau`. If either goes negative you do not get an exception; you get the logarithm of a negative number, a nan, and it propagates silently through a billion dynamic-programming cells with nothing pointing back at the cause. Three checks at construction time turn that into a message.

`LogTrans` holds the eleven log transition probabilities, computed once per scoring run rather than per cell. It has **no `d_e` and no `b_d` field**: the forbidden transitions are not representable, so later code cannot reach for them by accident, and a test asserts they stay absent. The D row reads `gD + (1-gD) = 1` with no termination term — the `D -> E` exclusion showing up as arithmetic.

Quality handling. `quality_table` precomputes emissions for all 94 Phred values, because the recursion indexes by quality at every cell. The clamp at `eps = 0.75` is load-bearing: at `eps = 3/4` the match probability `1-eps` and the mismatch probability `eps/3` are both 0.25, so `s = 0`; above it the model claims the sequencer prefers wrong bases, which is not an error model. HiFi BAMs write Q0 to mean "unknown", and unclamped that gives `log(1-eps) = -inf`, sinking the entire read. Q0 and Q1 clamp; from Q2 up nothing is touched, so real data never meets it.

5.3 The coordinate system

The one design decision everything else follows from. The dynamic program runs over `(u, v)`:

u read bases emitted so far, 0 .. L v ABSOLUTE template index reached, 0 .. T

Absolute, not relative to the read's start. If `v` counted from the origin you would need a separate matrix per origin. Absolute indices mean **one matrix holds every origin at once**, which is what makes the sum over origins cost `O(L*T)` instead of `O(L*T*T)`.

```
cell        meaning                                          valid when
--------    ---------------------------------------------    ----------
fM[u,v]     last state M, aligning r[u-1] to t[v-1]           u>=1, v>=1
fI[u,v]     last state I, emitted r[u-1], consumed t[0..v-1] u>=1, v>=0
fD[u,v]     last state D, consuming t[v-1]                   u>=1, v>=1
```

`fD[0, :]` is never written and stays at -inf for the whole run, because D is reachable only from M and M needs `u >= 1`. **That untouched first row is the `B -> D` exclusion.**

5.4 Seeding the start

The subtlest arithmetic in the file. The two begin edges differ by exactly one position: `B -> M` at origin `o` aligns `r[0]` to `t[o]`, so one template base is consumed and `v = o+1`; `B -> I` emits `r[0]` and consumes nothing, so `v` stays at `o`.

```
start_m[lo + 1 : hi + 1] = tr.b_m
start_i[lo : hi]         = tr.b_i
```

The offset between the two slices is that one-position difference, written down.

5.5 Two extents, not one

A window has a **template** extent and an **origin** extent, and they are not the same interval.

The template extent must be long enough to hold the read plus slack for indels and soft-clipping. The origin extent is the set of start positions this call is responsible for. The `origin` argument controls the second independently:

```
origin=None       every position in the template
origin=(lo,hi)    a half-open range; the template may extend past hi
origin=o          exactly one position
```

Why this needs saying: the model's sum runs over every origin in the unit, while a padded window around one alignment covers only that alignment's neighbourhood. So one call is in general one **term**. Terms combine by log-sum-exp, valid only when origin ranges are disjoint. Template extents must overlap — a read starting near the right of one range still needs sequence to align into — while origin extents must not, or shared origins are counted twice and every repeat is silently inflated, in the direction that looks like a real finding.

5.6 The main loop

For each read base `u`:

```
hit    = (tmpl == base) & (base != N) & (tmpl != N)
emit_m = np.where(hit, qt.log_match[q], qt.log_mismatch[q])

# M: all three predecessors are at (u-1, v-1), the only diagonal move
fM[u, 1:] = emit_m + combine(fM[u-1,:T] + m_m,
                             fI[u-1,:T] + i_m,
                             fD[u-1,:T] + d_m)

# I: predecessors at (u-1, v) -- same v, an insertion eats no template.
# fD is absent: the graph has no I-to-D edge in either direction.
fI[u] = log_insert + combine(fM[u-1] + m_i,
                             fI[u-1] + i_i)

# D: depends on the SAME row at v-1, so it is a left-to-right scan
for v in range(1, T+1):
    prev = combine(fM[u, v-1] + m_d, prev + d_d)
    fD[u, v] = prev
```

The ordering is load-bearing. M and I read row `u-1`; D reads row `u`. So M and I are computed first, then D. Reversing that would have D read an all-minus-infinity `fM[u]`, which fails silently rather than crashing.

M and I vectorise over `v` because row `u-1` is already complete. D cannot, because of that left-to-right dependency; it runs as a scalar scan carrying `prev`.

One body, two semirings. `combine` switches between log-sum-exp (forward) and maximum (Viterbi), so both share one recursion. Deliberate: the homopolymer test of section 3.5 compares the two, and separate implementations could drift apart and report the difference as a modelling result rather than a bug.

5.7 Finishing

```
terminal = logaddexp(fM[L], fI[L])     # distribution over END positions
logp     = logsumexp(terminal)         # free end: absorbs trailing padding
```

`fD` is excluded from `terminal`, and **that exclusion is `D -> E` being forbidden**. Arithmetic, not a check.

5.8 Viterbi and traceback

Start at the best end cell, choose M or I by whichever is larger, then walk backwards, each state moving the indices by what it consumes: M by `(u-1, v-1)`, I by `(u-1)`, D by `(v-1)`. The walk stops when a back-pointer says START. On exit `v` sits on the origin, and both seeding conventions land correctly.

Viterbi is never used for scoring. It exists to locate a read, to give the homopolymer test something to compare against, and to report where the best path started — the left-edge diagnostic.

5.9 Diagnostics

`edge_mass(margin)` is the fraction of terminal mass ending within `margin` of the template's right edge. A large value means the read wanted to run past the window and the sum may have been truncated.

Read it carefully, though: in the two-copy example of section 1.5 the edge mass is 0.50, because the read placed on the second copy legitimately ends exactly at the template's end. That is not an error — it is the diagnostic correctly saying "a placement touches the boundary, so there may be more copies beyond it". Which, for a repeat, is exactly the question.

It covers the right edge only. For the left, compare `viterbi().origin` against the same margin. An exact distribution over origins needs a backward pass, deferred rather than built speculatively.

6. What the tests establish

Twenty-two tests across two files. Three carry most of the weight.

Exhaustive path enumeration (the anchor). For `L <= 4` and `T <= 6`, a separate routine walks the transition graph directly — not reusing any recursion — and sums whole paths one at a time. The dynamic program must match. Across 72 configurations spanning free and fixed origins, free and full template, the worst deviation is **3.55e-15**. Because that enumerator has no `B -> D` and no `D -> E` edge, it pins both exclusions as a side effect.

Windows compose over disjoint origin ranges. Partitioning a three-copy unit's origins across three calls, each with a template running to the unit's end, reproduces the whole-unit pass to 1e-12. Adding an overlapping range visibly inflates the total. This is the contract a faster origin-anchored kernel will have to satisfy.

Copy number. Three exact tandem copies score exactly `log 3` above one copy. Proposition 4 in miniature, and a standing regression test: if it breaks, the method has gone blind to collapsed arrays.

The rest: one mismatch costs exactly `s`; log-likelihood falls monotonically with edit distance; the homopolymer ratio is exactly 5; a positive `tau` shifts the score by exactly the predicted amount; Viterbi never exceeds forward; Viterbi finds a planted read at the right coordinates; edge mass flags a read hanging off the window; the origin range rejects malformed bounds.

6.1 An instructive near-miss

Two test premises of mine were wrong, and the reason will recur.

I expected that padding a template with extra flanking sequence would leave the score unchanged, since the added origins explain a random read only through about `L` mismatches, which is `e^-200`. Measured, the score moved by 9.9e-5.

The cause was not spurious origins. The random pad happened to begin `TTT`, extending the

read's final `T` into a homopolymer run. The read's last base could then match one, two or three positions further right, after one, two or three deletions — worth `(1 - gD^k) * dD`. Exactly the degeneracy of section 3.5, in a place I was not looking for it.

The honest statements: padding is score-invariant to machine precision **when gaps are disabled**, and with gaps enabled it adds a non-negative, data-dependent amount bounded by the total gap-open mass `dD/(1-gD) + dI/(1-gI)`. The tests now assert both.

The general lesson: **the sum over origins is genuinely sensitive to sequence just outside the window whenever a homopolymer straddles the boundary.** Bounded and small, but real, and `edge_mass` will not catch it because the mass is interior. If a future fast kernel disagrees with this oracle at the 1e-4 level, check this first.

7. Limits, and what is not built

7.1 Hard limits of the current code

Memory binds first. Three float64 matrices of `(L+1) x (T+1)`. A 15 kb read against a 100 kb unit is 12 GB *per matrix*, so 36 GB. The oracle is usable to roughly `L*T <= 1e8`, about 2.4 GB — fine for a 1 kb read against a 100 kb template, or a 15 kb read against 6 kb. Production scale needs banding and row recycling.

Speed. `O(L*T)` with the D scan in Python: a few seconds at `L*T` around 3e6. Acceptable for an oracle.

7.2 Not implemented

- The normaliser `Z(L)`. With context-independent gaps it cancels from the comparison, so scoring does not need it yet.
- A generative sampler. The model can be run forwards to *produce* reads, which would give exact self-consistency tests and a parameter round-trip.
- Context-dependent gap parameters, where HiFi errors actually concentrate — the pair-HMM analogue of a profile's position-specific parameters (section 2.6). This is the expensive one: about forty parameters, and it resurrects `Z`.
- Any connection to real data: no read store, no origin enumeration, no FASTA adapter, no strand handling at the boundary. Nothing in `dp.py` reverse-complements anything; the caller supplies the strand-oriented template.
- Parameter estimation. The current values are placeholders of roughly the right order for HiFi, not measurements.

7.3 Decisions still open

Scope. Curated loci or a genome-wide survey? This sets whether the whole-unit pass is affordable as-is or whether an origin-anchored kernel is mandatory. The largest single fork in the plan.

Compiled kernel. numba pins llvmlite tightly against numpy and Python versions, and the project `venv` is a symlink into mambaforge base, so installing there affects every other project. Alternatives: an isolated environment, or the kernel in C behind ctypes. Nothing before the compiled kernel needs this decided, but it should be decided rather than discovered.

Flanking states. Section 2.5 — whether to add HMMER-style states that absorb read bases outside the segment, instead of excluding boundary reads by the containment rule. It would recover a third of the reads at a 45 kb locus, but it changes what the score means and interacts with the exposure term. Worth a decision, not a default.

Insertion emissions. The flat 1/4 is crude for HiFi, where inserted bases sit inside homopolymers and are nearly determined by context. Whether that matters enough to justify the context-dependent model is a measurement, and the simulation work is what would measure it.

8. Summary

The pair HMM answers one question: **how probable is this read, if it came from this exact place in this assembly?** Four inputs — read, qualities, template, five parameters — and one number out.

It answers by adding up every way the sequencer could have produced the read from that template, rather than picking the best one, because the best-path answer is biased differently for the two assemblies being compared.

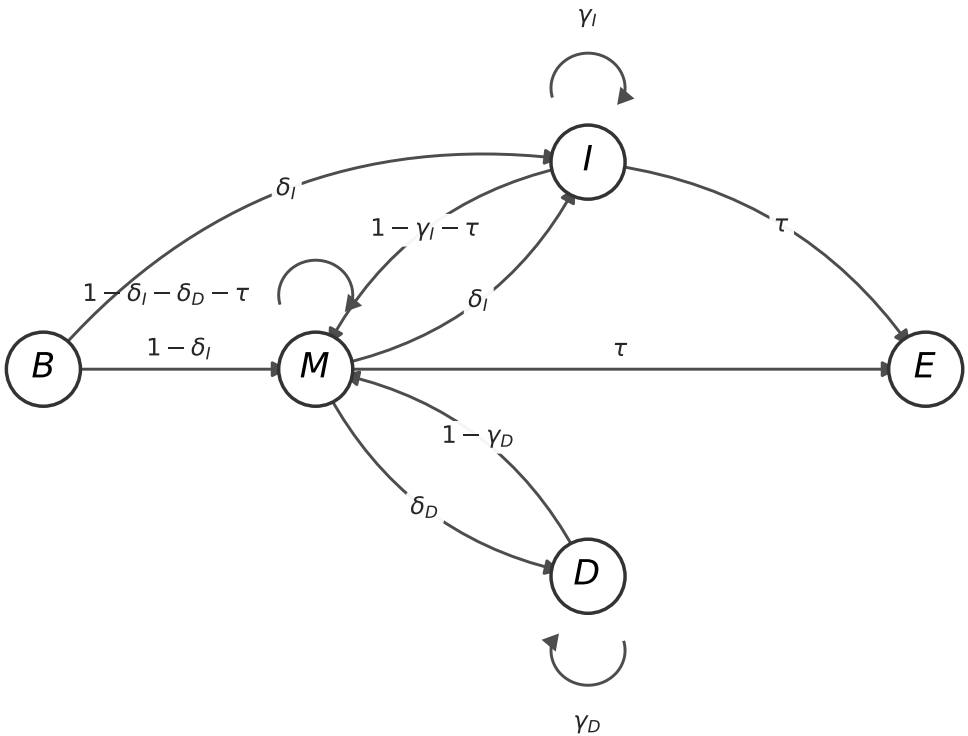
Unlike a profile HMM, the sequence being compared against is an *argument*, not part of the model. That is what makes it usable here at all: every candidate position gives a different template, and there is nothing to train on.

The place is an origin: a contig, a position, a strand. Summing the answer over all origins in the unit gives the quantity the likelihood model needs, and that sum is where copy number lives — three copies means three terms means `log 3`.

What exists today is the model, exactly, with no approximations, verified against exhaustive enumeration at 3.55e-15. It is slow and it is not connected to any real data. Both are deliberate: the slow exact version is the thing every faster version will be checked against.

The pair HMM transition graph

Two edges are deliberately absent: B→D and D→E. Both remove redundancy, not real possibilities.



M consumes one read base and one template base I consumes a read base only D consumes a template base only